

INDEXING IN DBMS

Indexing is technique used in database Management system (DBMS) to speed up the retrieval of rows from a table.

Instead of scanning the entire table every time a query is executed, DBMS uses an index to quickly locate the required data.

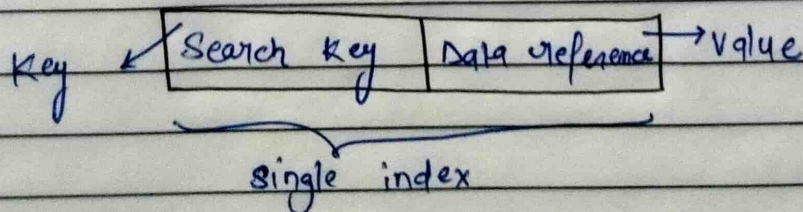
It works similarly the index of a book:

- Without an index - You search page by page.
- With an index - you directly jump to the required page.

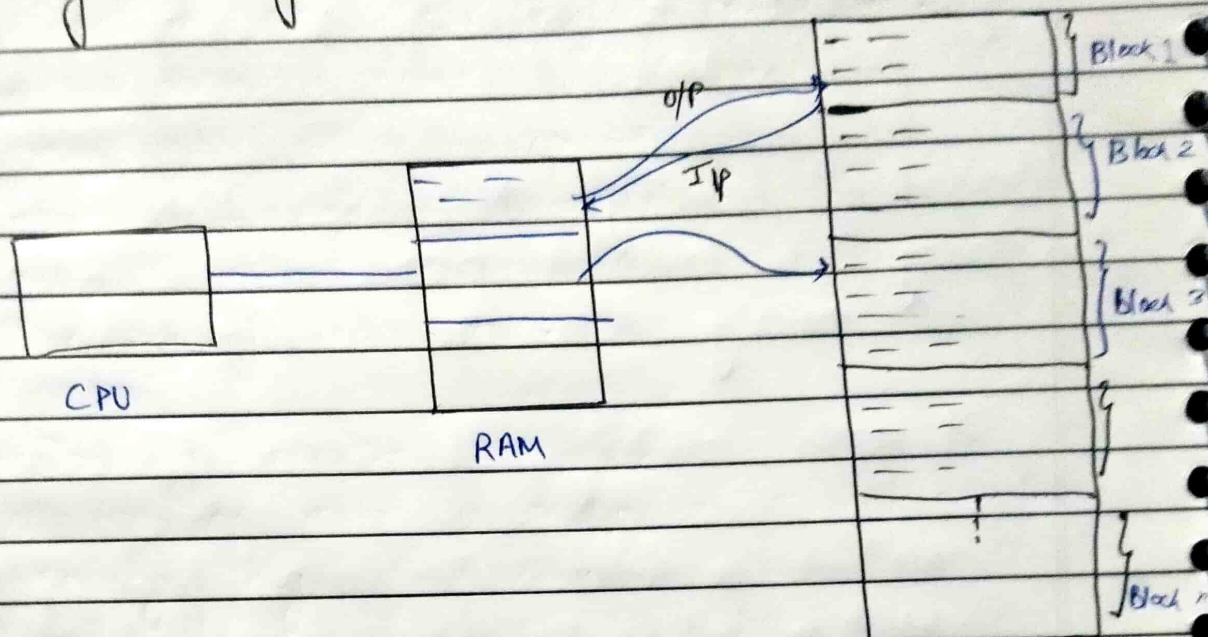
Definition :-

- An indexing is a data structure that stores:
- the value of one or more attributes (columns)
 - along with pointer/references to the actual row in the table.

Structure :-



→ Why indexing is needed :-



Suppose a table contains
10 lakh record (row)

Secondary
Storage

Query :-

`SELECT * FROM Student
WHERE Roll_No = 105;`

Without indexing:

- DBMS checks every row one by one.
- This is called **FULL TABLE SCAN**.
- Time complexity $\approx O(n)$.

With indexing:-

- DBMS directly reaches the required row.
- Much faster.
- Time complexity $\approx O(\log n)$.

→ Advantages of Indexing :-

1) Faster Data Retrieval :-

Queries executed quickly

2) Reduces Disk I/O :-

Less number of data pages are accessed.

3) Improved Search performances :-

Useful for :

- WHERE clause

- ORDER BY

- GROUP BY

- JOIN operation

4) Helps in Sorting :-

Indexed columns are already organized.

→ Disadvantages of Indexing :-

1) Extra Storage required :-

Indexes occupy additional memory

2) Slower Insert/Update/Delete :-

Whenever data changes, indexes also need updating.

3) Maintenance cost :-

Too many indexes reduce performances.

→ File organization in indexing :-
File organization refers to how data and indexes are physically stored in memory or on disk.

Common types of file organization :-

1.1. Sequential (ordered) file organization :-
In this type of organization, the indices are based on a sorted ordering of the values. These are generally fast and a more traditional type of storing mechanism.

These ordered or sequential file organization might store the data in a dense or sparse format.

1.1.1. Dense index :-

A dense index is an indexing technique in which every search key value in the data file has an index entry.

This means :-

- each record is represented in the index table.
- the index directly points the exact record.

idea :-

Suppose a table contains 100 records.

In dense indexing :-

- there will also be 100 index entries.

Structure of Dense index :-

Student table :

Roll-No	Name
101	Ram
102	shyam
103	Mohan
104	Aman

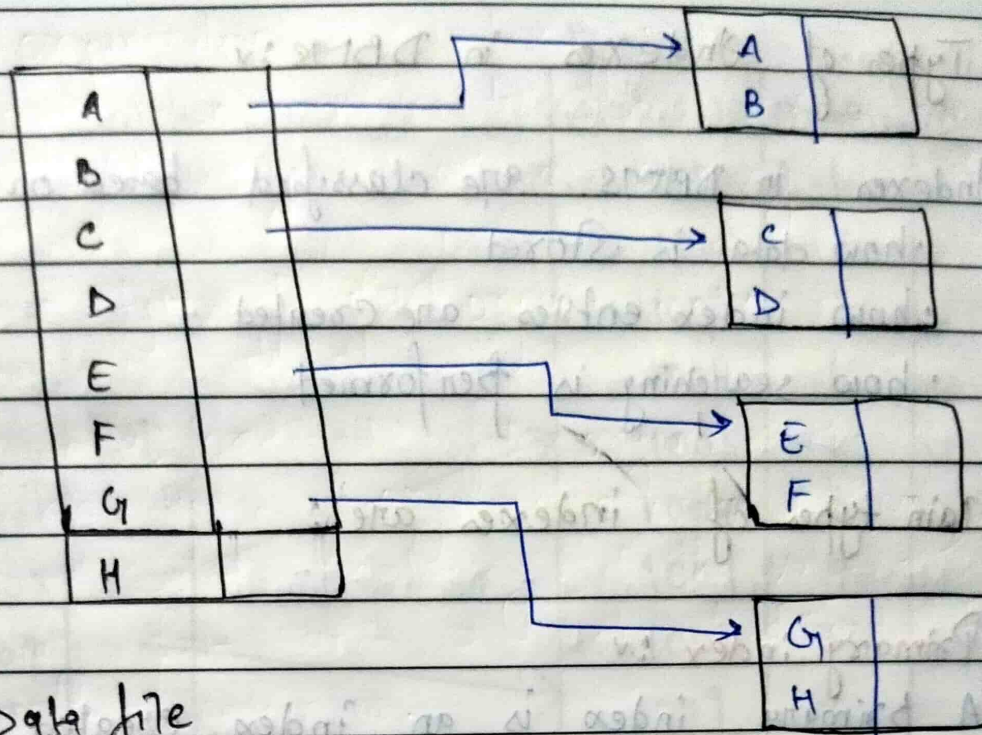
Dense index table

index key	Pointer
101	Address of record 101
102	" " " 102
103	" " " 103
104	" " " 104

1.2. Sparse index :-

A sparse index is an indexing technique in which index entries are created only for some records, not for every record.

$$\text{Access cost} = \log_2(n) + 1.$$



Data file

Index record

2.7 Hash file organization :-

Use hash funcⁿ , we learn later.

→ Types of Indexes in DBMS:

Indexes in DBMS are classified based on:

- how data is stored
- how index entries are created
- how searching is performed

Main types of indexes are:

1. Primary index:

A primary index is an index created on the primary key of a table.

The data records are physically stored in sorted order according to the primary key.

Primary key	Block pointer
-------------	---------------

A primary index is an ordered file whose records are of fixed length with fields. The first field of the index is the primary key of the data file in an ordered manner, and the second field of the ordered file contains a block pointer that points to the data block where a record containing the key is available.

Primary key		Block 1		
Student No.	Block pointer	Student-id	Branch no	Data
1001		1001	1	
1005		1002	2	
1009		1003	2	
1013		1004	1	
---	---	1005	1	
		1006	3	
		1007	4	
		1008	3	
		1009	3	
		1010	3	
		1011	4	
		1012	1	
		1013	2	
		1014	2	
		-	-	
		-	-	
		-	-	

Primary index

→ Type of Primary indexing :-

1. Dense indexing :

In dense indexing has an index entry for every search key value in the data file.

This approach ensures efficient data retrieval but requires more storage space.

No. of Index entry = No. of DB record.

Data file Block 2 Block 3

iii) Sparse Indexing :-

Sparse indexing involves having fewer index entries than records. The index entries point to blocks of records rather than individual records. While it reduces storage overhead, it may require additional disk access during retrieval.

No. of index entry = No. of block

2.4 Clustering indexing in database :-

Clustering indexing is a database indexing technique that is used to physically arrange the data in a table based on value of the clustered index key.

A clustering index is created when records are physically stored according to a non-key attribute in a order.

Records having same value are grouped together.

With the clustered index, the database can more efficiently retrieve data because it doesn't have to scan the entire table to find the data it needs, instead it can use the clustered index to quickly locate the data.

Clustering index :-

S_Name	Block pointer
Akash	B1
Ayush	B1
Piyush	B1
Vivek	B2
Aditya	B2

S_id	S_Name	
1	Akash	} B1
2	Akash	
3	Ayush	
4	Piyush	
5	Vivek	} B2
6	Vivek	
7	Vivek	
8	Aditya	
9	Aditya	} Bn
⋮	⋮	
⋮	⋮	

Use when :-

select * from Student
where S_Name = 'Vivek';

Disadvantages :-

- Increase Complexity
- Reduce update performance
- limited to one clustered index.

Note :-

Clustering indexing may not always be the best choice for every situation. For eg; if our data is constantly changing and new data is frequently added to the table, clustering indexing may not be the best choice. This is because the database has to physically ^{or} rearrange the data every time new data is added to the table, which can be time-consuming and negatively impacts performance.

3) Non-clustered / Secondary indexing :-
It is of two types :-

(i) Secondary key indexing :-

A secondary key indexing is an index created on a non-ordering attribute of a file. The data records are ~~not~~ not physically stored in the order.

A non-clustered index does not change the physical data order.

In simple words :-

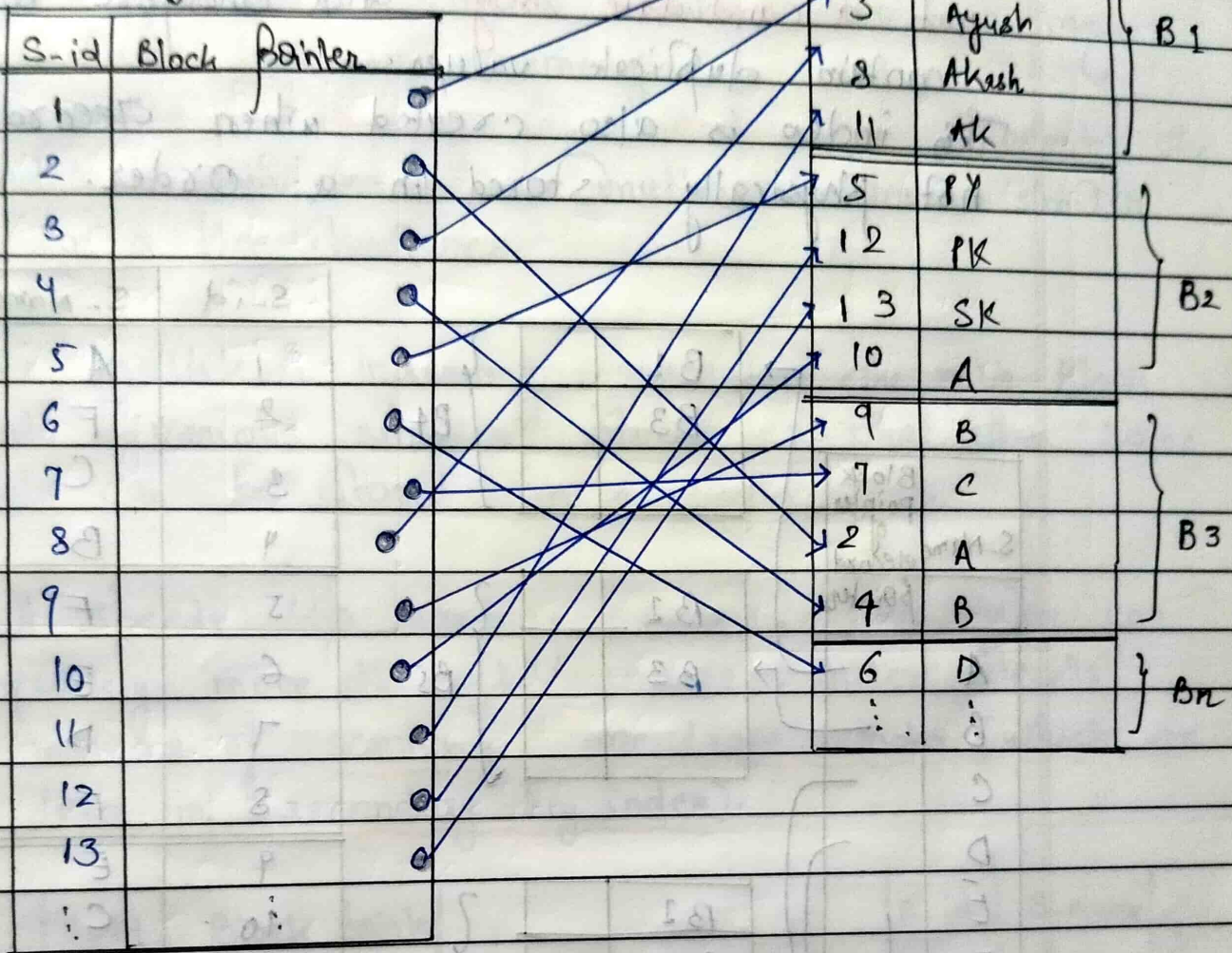
A secondary key indexing is created when records are not physically stored in a order, ~~arranging the data in order~~

A secondary key index is an index created on primary / super key of a table.

It resolves the problem that occurs in clustering index (which is to rearrange the data every time whenever a new data enter)

But the problem is, we have to make dense indexing.

Secondary key indexing :-



(ii) Secondary Non-key indexing :-

A secondary non-key indexing just tells us where the data lies i.e. it gives us a list of virtual pointer or references to the location where the data is actually stored.

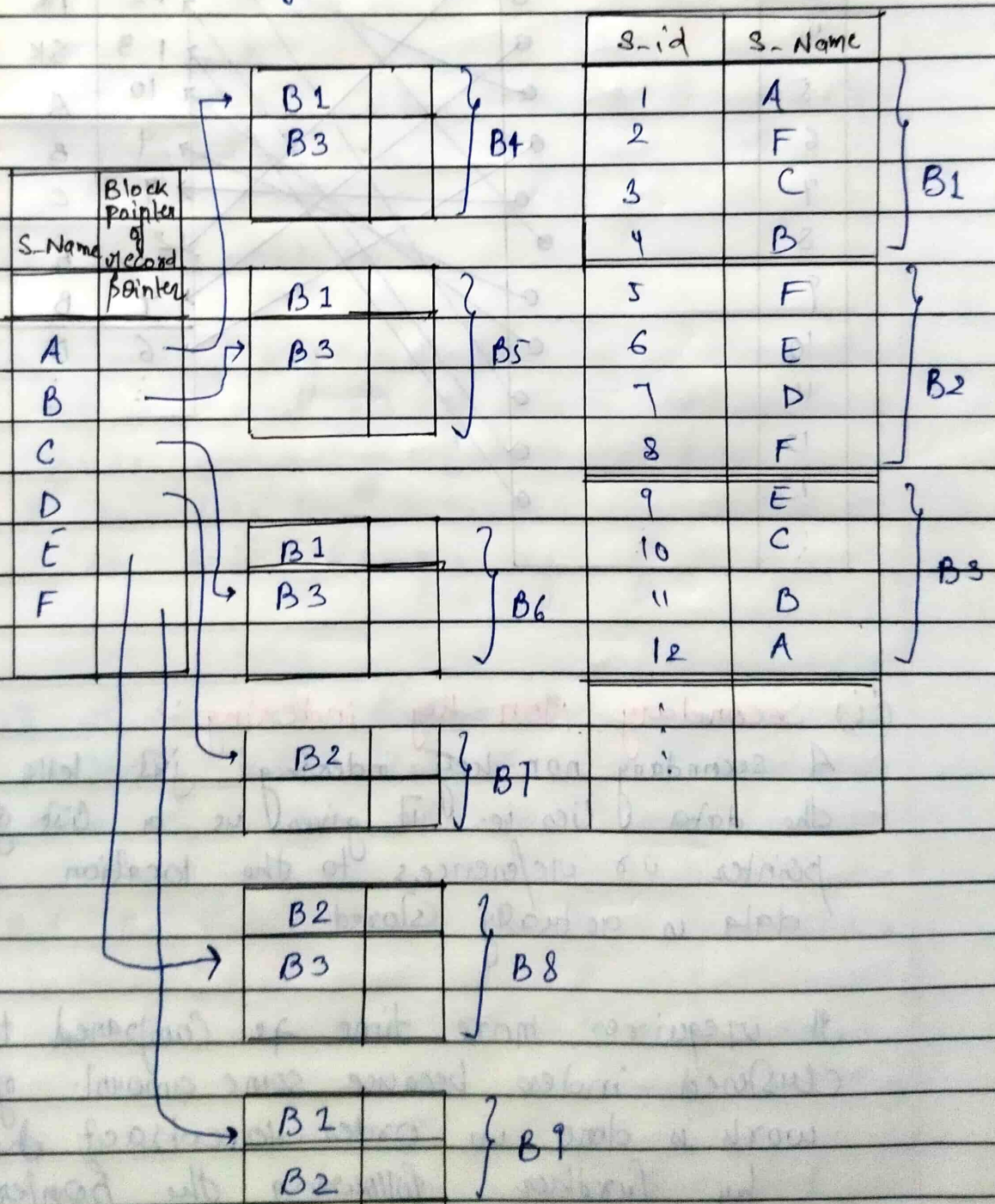
It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer.

Note:-

It mainly needs when multiple duplicate value occur which made him differ from multilevel where its need when indexes became too large

A secondary non-key index is a secondary index built on a attribute that is **not a candidate key** and therefore may contain duplicate values.

This index is also created when records are not physically stored in a order.



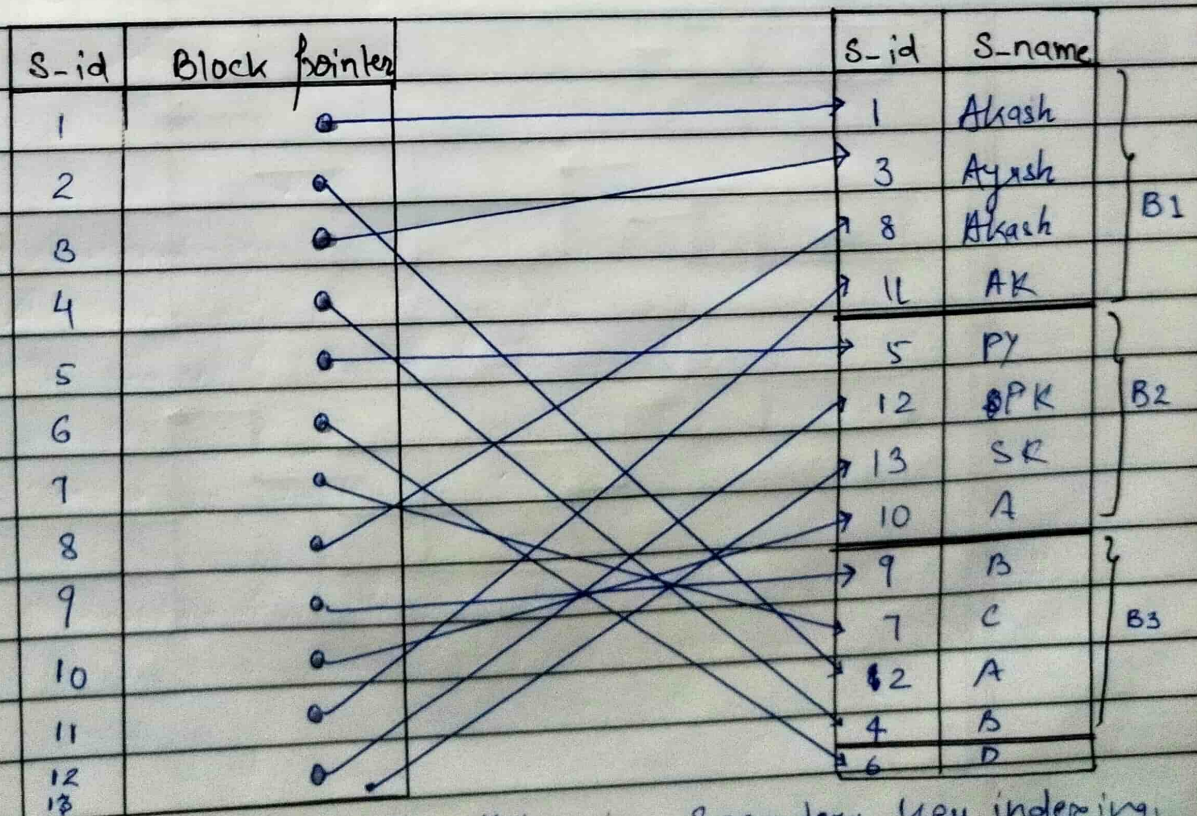
→ Multilevel Indexing :-

Multilevel indexing is a technique used when a single-level index becomes too large to store in main memory.

To make searching faster and more efficient, the index is divided into multilevel levels similar to a tree structure.

The multilevel indexing separates the main block into various smaller blocks so that the same data can be stored in a single block.

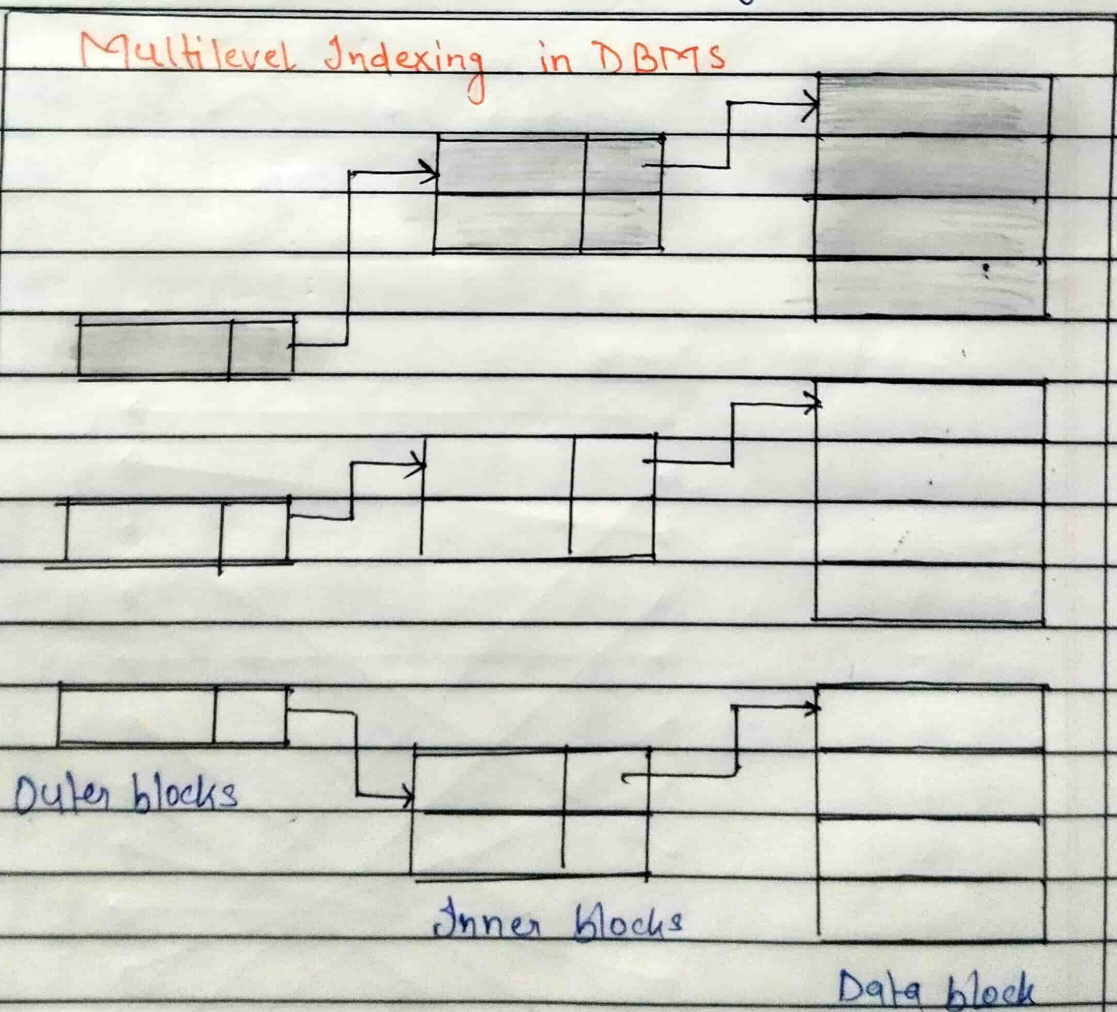
It speeds up searching because the system can quickly move through smaller index levels instead of scanning one large index (which we learn in secondary key index).



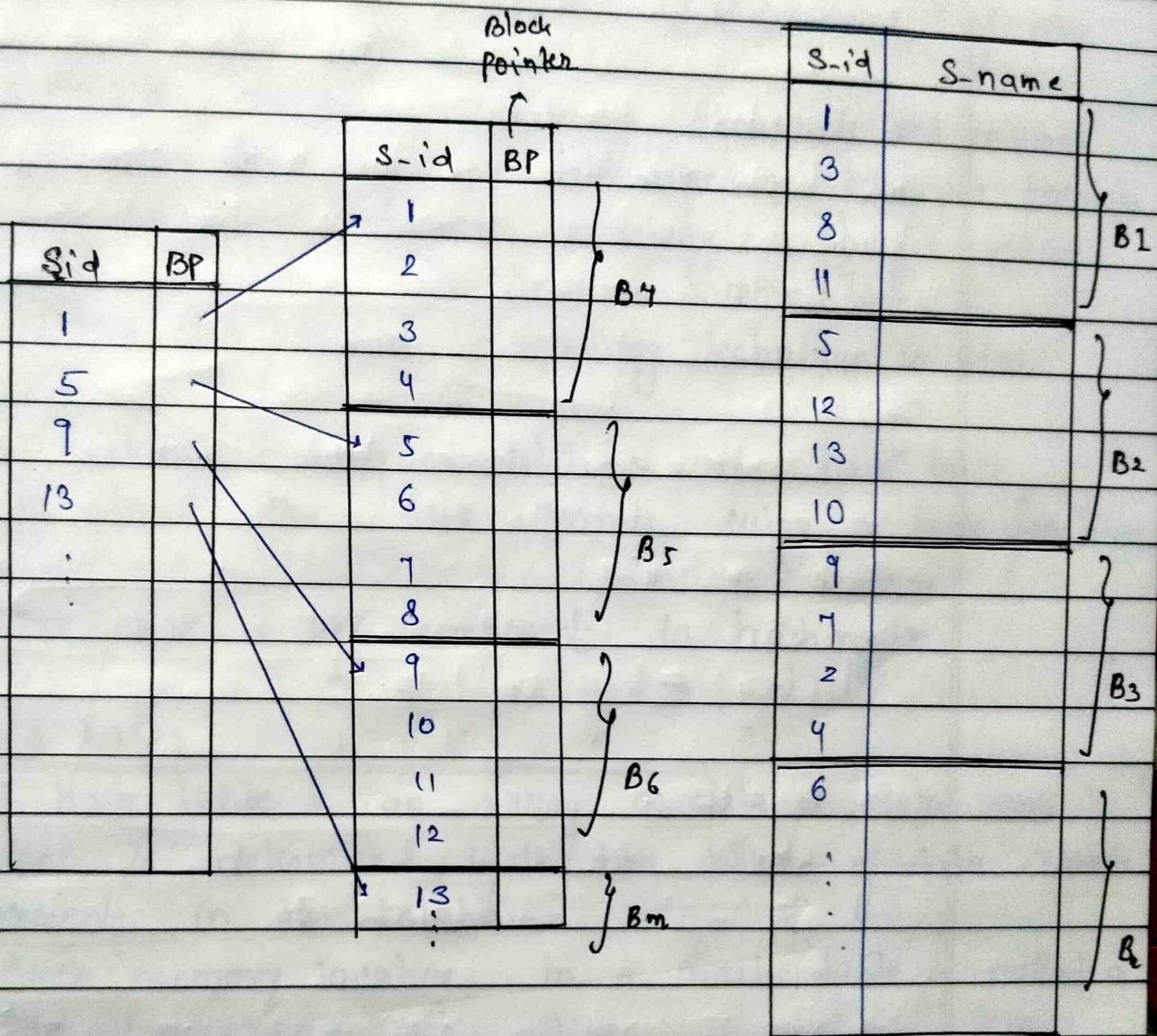
We have done this thing in Secondary key indexing.

→ Needs of Multilevel indexing :v

- Single-level indexes become too large for big databases and require many disk accesses.
- Multilevel indexing **reduces disk I/Os** by creating indexes on top of indexes.
- Dividing the index into smaller blocks ensures every block fits in one disk block, reducing access time.
- With fewer blocks to scan, the system can retrieve results more quickly.



This multilevel we apply in our secondary key indexing example.



→ Types of Multilevel Indexing is

- ① B-Tree
- ② B+Tree

{ we will later after hashing }

→ Hashing in DBMS :-

Hashing in DBMS is a technique to quickly **locate** a data record in database irrespective of the size of the database.

For larger database containing thousands and millions of records, the indexing data structure technique becomes very inefficient because searching a specific record through indexing will consume more time. So counter this problem hashing technique is used.

The hashing techniques utilizes an hash table to store the data records using a hash function.

→ There are 3 key components in hashing :-

• Hash table:

A hash table is an array or data structure and its size is determined by the total volume of data records present in the database.

Each memory location in a hash table is called a **'bucket'** or **hash indices**.

~~XXXXXXXXXXXX~~

• Bucket: /

A bucket is a memory location (index) in the hash table that stores the data record.

~~Hash function :-~~

~~A hash function is a mathematical equation or algorithm that takes one data record's primary key as input and computes the hash index as output.~~

• Hash function:~

A hash function is a mathematical ^{equation or} function that converts a key into a hash value (index/address) where the data will be stored in a hash table.

This hash function is the most crucial component that determines the speed of fetching data.

Eg:

Suppose the hash table size is 10.

Hash function:~

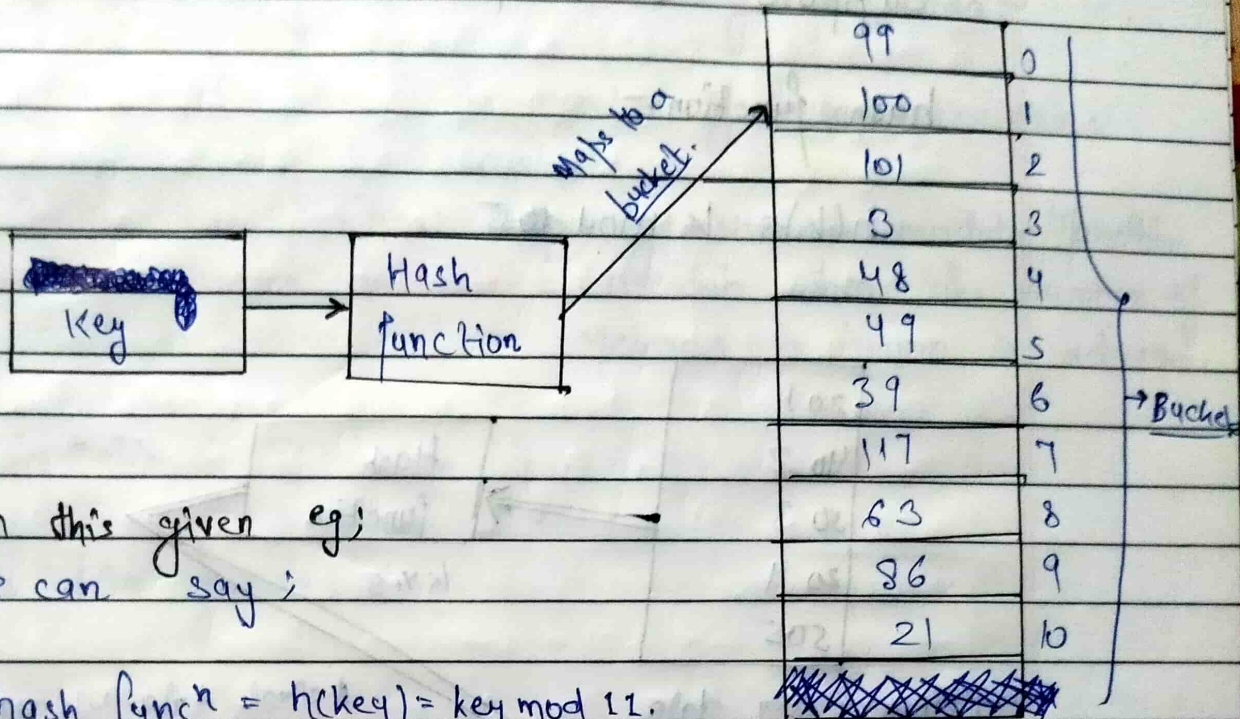
$$h(\text{key}) = \text{key} \bmod 10.$$

For key = 123:

$$h(123) = 123 \bmod 10 = 3$$

So the record is stored at index = 3.

Key	Hash value	Index
123	3	3
145	5	5
208	8	8



In this given eg;
we can say;

$$\text{hash func}^n = h(\text{key}) = \text{key} \bmod 11.$$

Hash table

→ Types of hashing in DBMS :-
There are four primary hashing techniques in DBMS.

1. Static Hashing :-

Static Hashing is a hashing technique in which the number of bucket remain fixed and the number of bucket does not change even if the database grow.

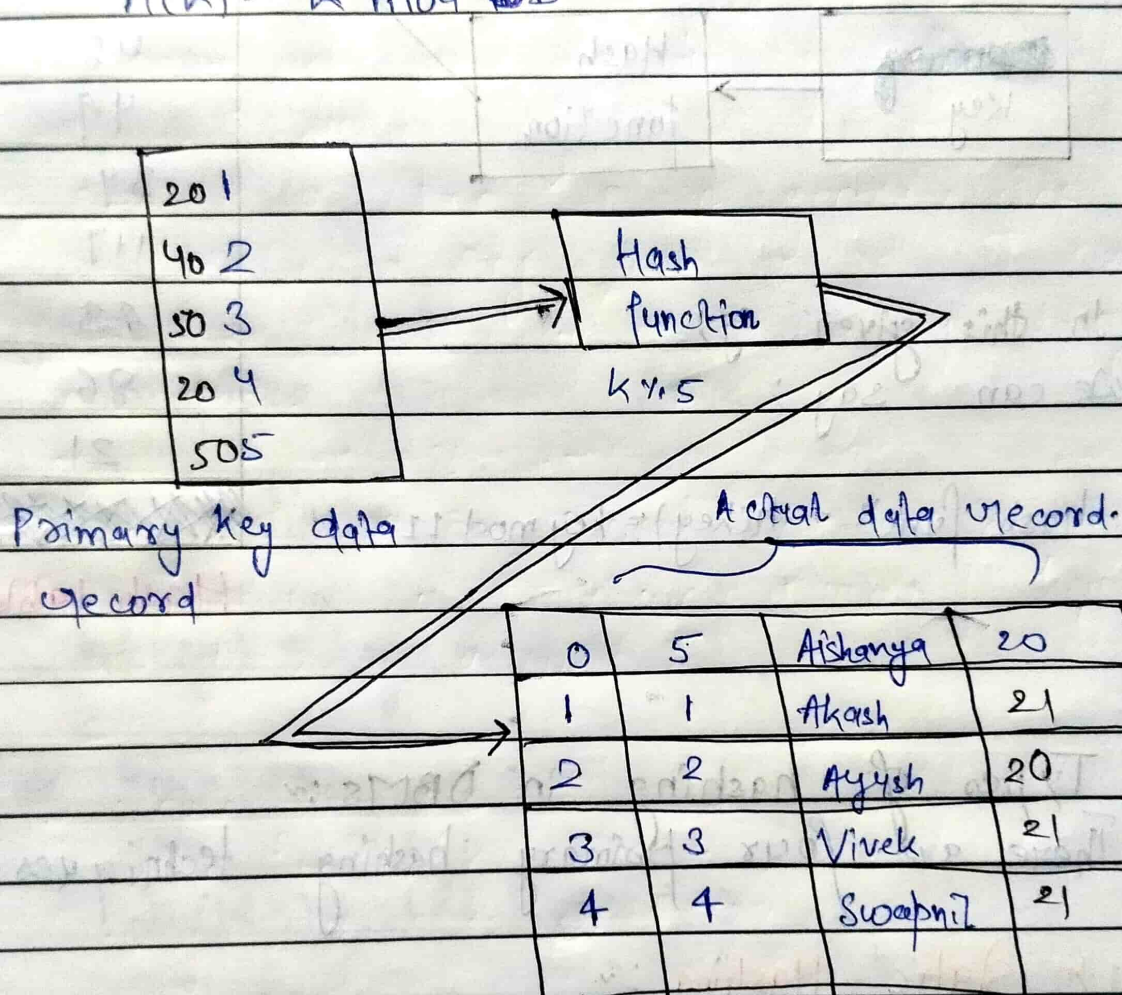
Eg; Student table :-

Sid	S-name	S-age
201	Akash	21
402	Akash	20
503	Vivek	21
204	Swarnil	21
505	Aishwarya	20

Suppose there are 5 buckets (0-4)

hash function:-

$$h(k) = k \bmod 5$$



The primary key is used as the input of the hash function and the hash function generates the output as the hash index (bucket address) which contains the address of the actual data record on the disk block.

→ Static hashing has the following properties:-

- **Fixed table size:** The number of bucket remains constant.
- **Simple hash function:** Typically uses a modulo function.
- **Best for known data size:** Efficient when the number of records is known in advance.

Limitation is

As data grows:

- (i) Collision increase.
- (ii) ~~data~~ leading to bucket overflows.

Eg for bucket overflows:-

Suppose we have five bucket (0-4) {fixed}

We have key:-

$$\text{hash func}^n = k \bmod 5.$$

102

105

103

108

For key 108, ~~this~~ this leads to bucket overflow.

To solve this problem of collision & bucket overflows technique used such as chaining and open addressing are used.

2* Chaining :- (open hashing)

Chaining is a mechanism in which the hash table is implemented where each bucket is of node type and can contain a long chain of linked list to store the data records.

So even if a hash function generates the same value for any data record it can still be stored in a bucket by adding a new node.

→ Here, collision are stored outside the table. This technique is used when number of key not known in advanced.

Eg: hash table size = 5

hash funcⁿ: $h(k) = k \% 5$

key: 10, 15, 12, 11, 14, 13.

key	Hash index ($k \% 5$)	Inserted at
10	0	Bucket-0 → [10]
15	0	B-0 → [10 → 15]
12	2	B-2 → [12]
11	1	B-1 → [11]
14	4	B-4 → [14]
13	3	B-3 → [13]
21	1	B-1 → [11 → 21]

hash table:-

Index	Linked List (bucket)
0	10 → 15
1	11 → 1
2	12
3	3
4	14

Properties :-

- All key that hash to that same index (10 → 15, 11 → 1) are stored in a **linked list** at that index.
- Separate chaining **avoids clustering** and make insertion easier.
- It is efficient.

This will give rise to the problem **bucket skew** i.e., if the hash function keep generating the same value again and again then the hashing will become inefficient as the remaining data bucket will stay unoccupied or store minimal data.

Eg:

- hash table size = 5
- Hash function: $h(u) = k \cdot u$
- key insert: 10, 15, 25, 30, 45, 50, 51

hash table

Index	Linked-List (bucket)
0	10 → 15 → 25 → 30 → 45 → 50
1	51
2	
3	
4	

3. Open Addressing (Closed Hashing):

This is also called closed hashing. This aims to solve the problem of collision by looking out for the next empty slot available which can store data.

It uses technique like:

Linear probing
Quadratic probing
Double probing hashing

Eg;

Hash table size = 7

Hash funcⁿ: $h(k) = k \% 7$

collision resolution: Linear probing

Insert the key: 50, 700, 76, 85, 92, 73.

Key	Hash($k \% 7$)	insert At	collision?	Final position (after probing)
50	$50 \% 7 = 1$	1	No	1
700	$700 \% 7 = 0$	0	No	0
76	$76 \% 7 = 6$	6	No	6
85	$85 \% 7 = 1$	1	Yes	2 (next slot)
92	$92 \% 7 = 1$	1	Yes	3 (after 1 & 2 are filled)
73	$73 \% 7 = 3$	3	Yes	4 (next slot after 3)

Hash table :-

Index	Value
0	700
1	50
2	85
3	92
4	73
5	---
6	76

After that; quadratic,
double hashing ~~are~~
~~these~~ is same as we
learn in DSA.

→ B-Tree :-

it is one of the type of ~~xxxx~~ Multilevel indexing.

A B-Tree (Balanced Tree) is a self-balancing tree data structure used to store and organize large amount of data stored data efficiently.

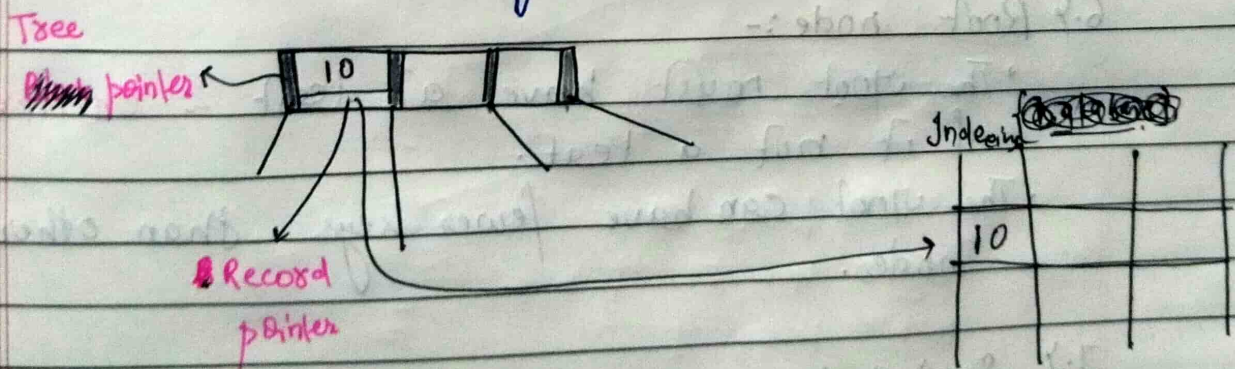
It is widely used in database, file system and storage system because it supports fast searching, insertion and deletion operation.

Why B-Tree :-

In a binary search tree, each node have at most two children. As data grows, the tree can become deep, increasing the number of accesses required to find data.

A B-Tree solves this problem by allowing multiple key and multiple children in each node, making the tree shorter and reducing the number of levels.

B-Tree of order $\rightarrow 4$.



→ Properties :-

Let m be the order of the B-Tree.

1. The value of m is decided based on disk block and key size.

2. Maximum children:

- A node can have at most m children.

3. Maximum keys:

- A node can contain at most $(m-1)$ keys.

4. Minimum children:

- Every internal node (except the root) must be at least $\lceil m/2 \rceil$ children.

5. Minimum keys:

- Every internal node (except the root) must contain at least $(\lceil m/2 \rceil - 1)$ keys.

6. Root node:-

- The root must have at least 2 children if it not a leaf.
- The root can have fewer keys than other nodes.

7. Sorted keys:-

- Keys within a node are sorted in sorted order.

8. Balanced - Tree :-

All leaf nodes are at the same level, making the tree height-balanced.

9. Child-Parent relation :-

If a node contains k keys, it has $(k+1)$ children.

10. Dynamic Growth and shrinkage:

The tree automatically adjusts through splitting and merging nodes during insertion and deletion.

Operation	Time Complexity
Search	$O(\log(n))$
Insert	$O(\log(n))$
Delete	$O(\log(n))$
Traverse	$O(n)$

1. Insertion operation :-

Step 1 :-

Always keeps keys in ascending order.

Step 2 :-

Insert only into a leaf node.

Step 3 :-

if overflow occurs :

1. Draw the overflowing node.
2. Circle or identify the middle key
3. Move the middle key to the parent.
4. Split the remaining keys into left and right child nodes.

Step 4 :-

if the parent also overflow, repeat the same process upward.

Q. 15, 8, 22, 5, 10, 18, 30, 45 of order 4.

Max^m keys = 3

Max^m children = 4

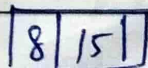
Min^m keys = 1.

Step 1: Always keeps key in ascending order:-

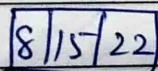
Insert 15:-



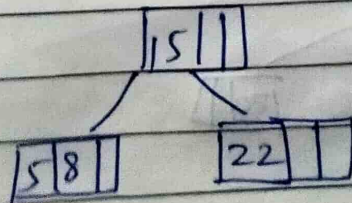
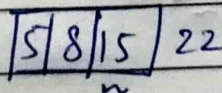
Insert 8:-



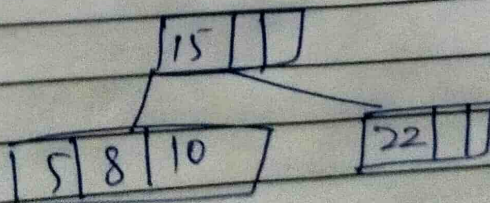
Insert 22:-



Insert 5:-

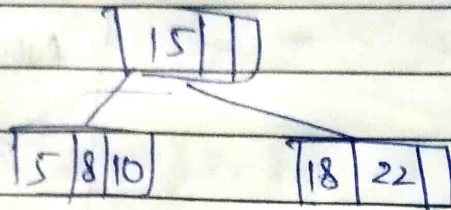


Insert 10:-

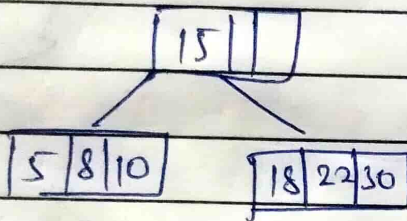


Note:- Key ko kisi 48th
Ascending order ke
list ke 5, 8, 10, 15, 18,
22, 25, 30 } Insertion
point karta hai aur Ans
ke liye 31yoga
par duniya karta hai
q k l hika.

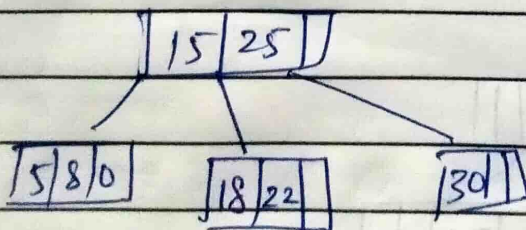
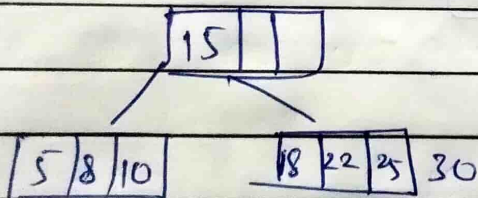
Insert 18 :-



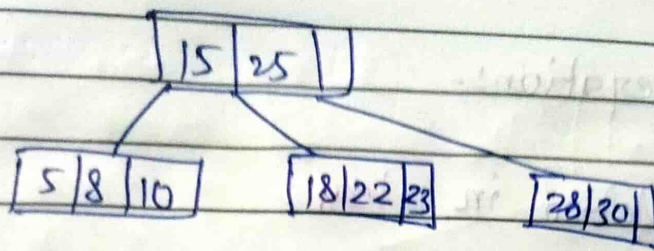
Insert 30;



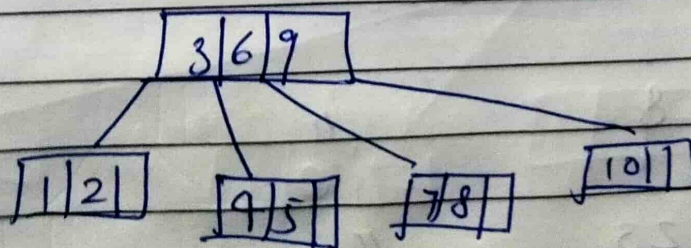
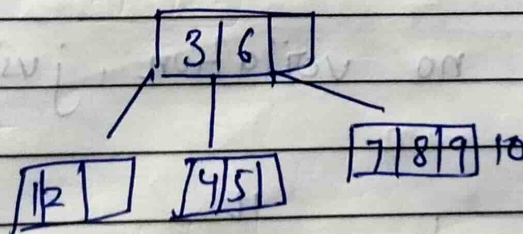
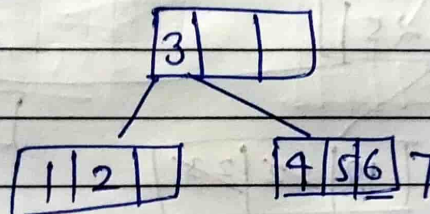
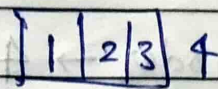
Insert 25 :-



Now, Any one said insert 23, 28



Eg Insert 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 :-



→ Deletion operation:-

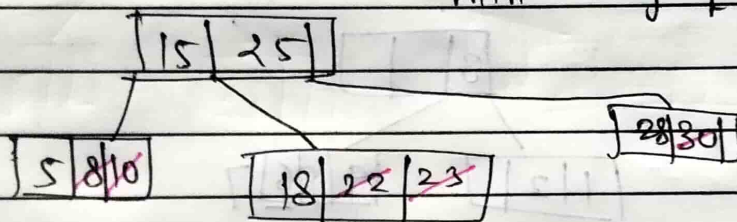
Case I :- Deletion in leaf:-

1. After deletion, if no violation of min keys, then no changes in tree.

Eg;

order $\rightarrow 4$

$$\text{min}^m \text{ key} = \lceil k/2 \rceil - 1 = 1$$



delete 10, no violation, just delete 10.

delete 22

11

delete 30

11

delete 8

11

delete 23

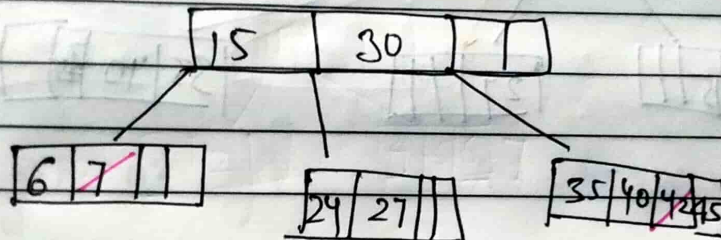
11

2. If violation of min keys, then borrow key from sibling (rotation through parent).

Eg: Order - 5

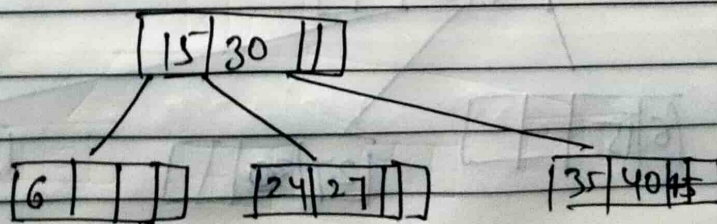
$$\text{min}^m \text{ key} \rightarrow \lceil k/2 \rceil - 1 = \underline{2}$$

Eg:



delete 42, No violation, just delete 42.

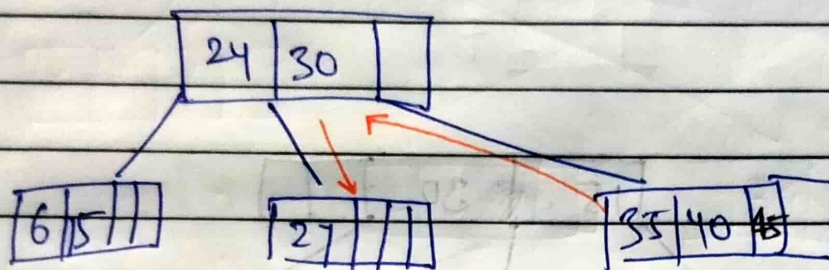
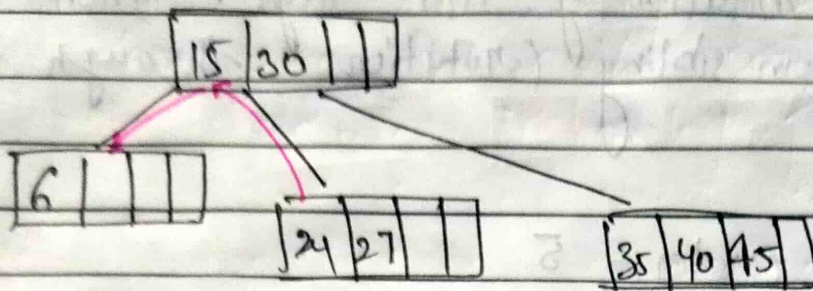
delete 7, violate min^m key (2).



Borrow from sibling

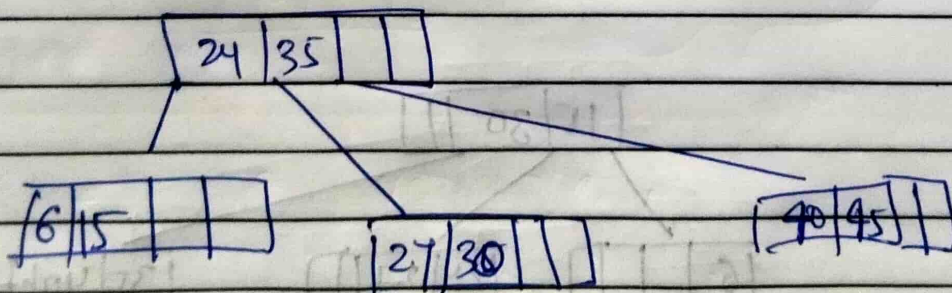
If 24 come near 6 then it violate B-Tree rule ($15 < 24$).

So, Rotation through parent.

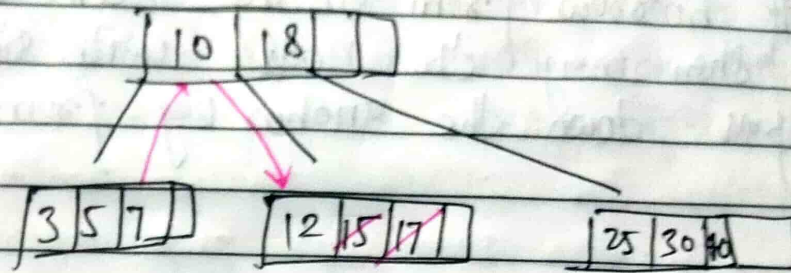


but here 27 violates min^h key rule.

Again Rotate through parent.



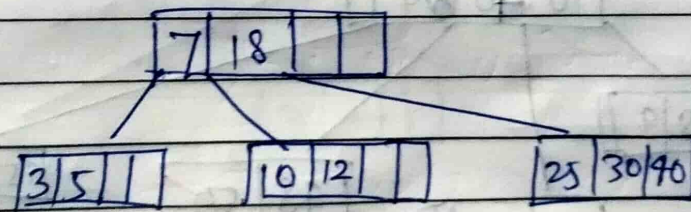
Eg: order-5 min key = 2



delete 17, no violation

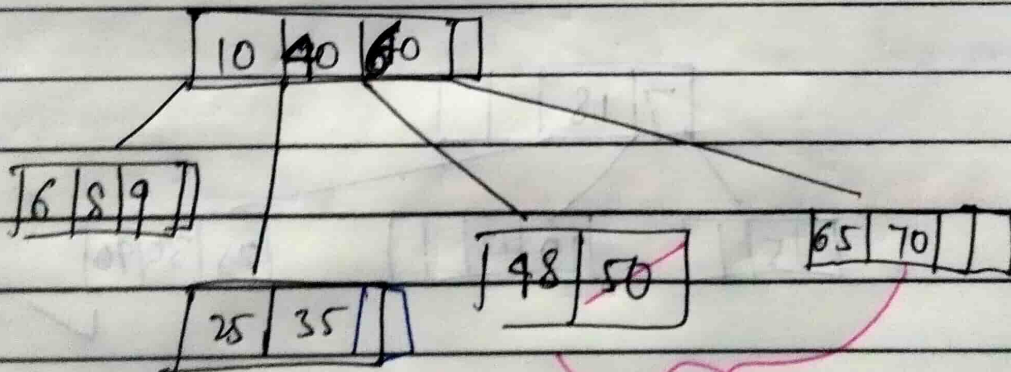
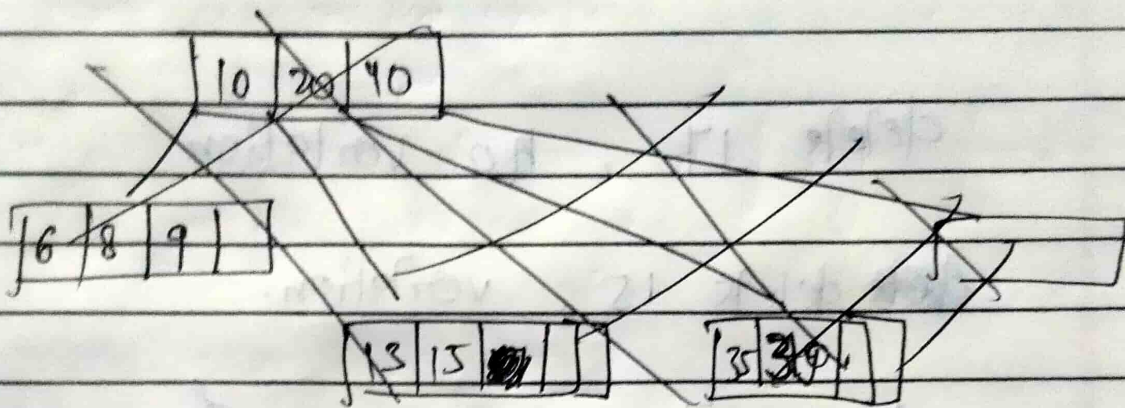
~~delete~~ delete 15, violation.

No w it can borrow from any side
Right or left, No left borrow left.



3.4 If borrow from sibling can't be possible then merge the node with sibling and pull down the anchor key from parent.

eg:

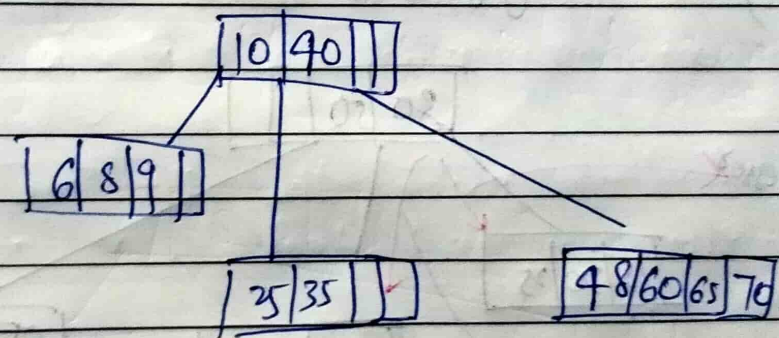
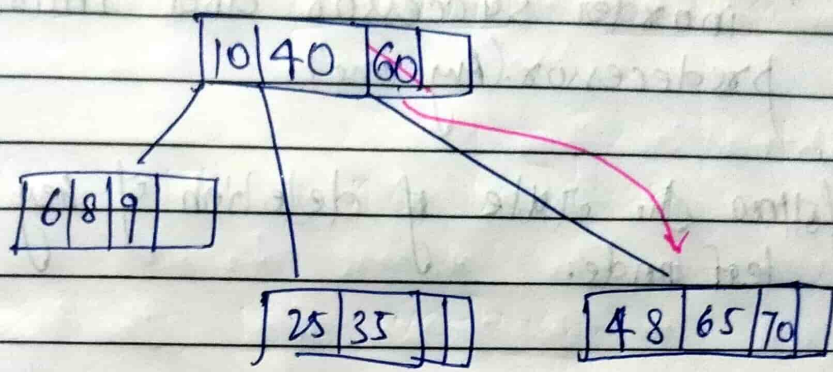


merge

delete 50. violation,

Cannot borrow key from either of immediate sibling.

We just merge the node of any one of the sibling and pull ~~the~~ down the anchor key from parent. (Merge Right most).



Inorder $\rightarrow$ Sorted sequence in ascending order

Date _____
Page _____

Case II :- Deletion in Internal Node :-

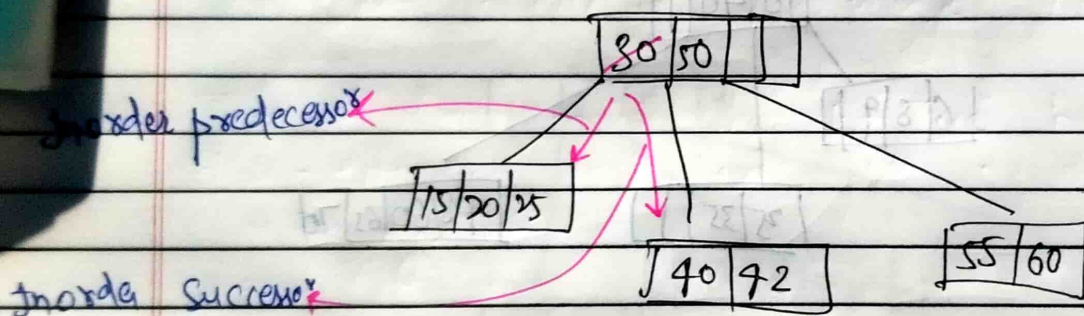
1) Replace the deleted value with ~~internal~~ inorder successor and inorder predecessor (Any one).

then,

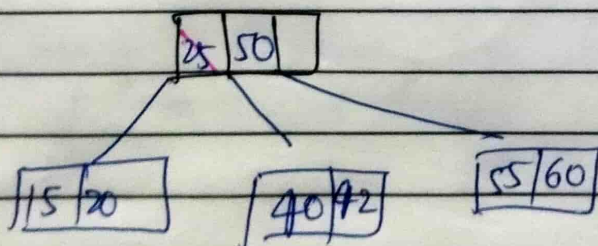
follow the rule of deletion of key from leaf node.

Eg ;

Order = 5

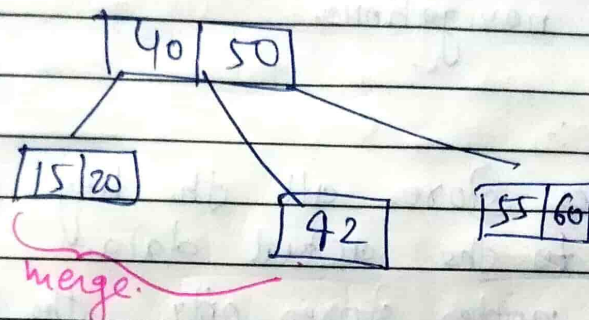


delete 30, let replace with inorder predecessor.



Again delete, 25

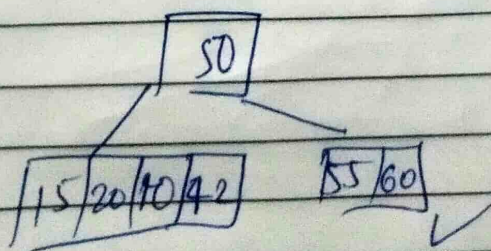
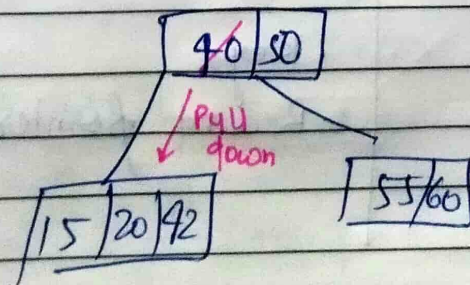
let replace with in order successor.



here 42 violate min^m key

So we follow the same rule in leaf node deletion,

here, we can't borrow from any of the immediate sibling so we merge it
let merge with left leaf node.



→ B+ Tree :-

It is an extended version of B-Tree, where all actual data is stored only in the **leaf nodes**, while internal nodes contain only keys for navigations.

→ Component :-

- leaf nodes stores all the key values and pointer to the actual data.
- Internal nodes stores only the keys that guide searches.
- All leaf nodes are linked together, supporting efficient sequential and range queries.

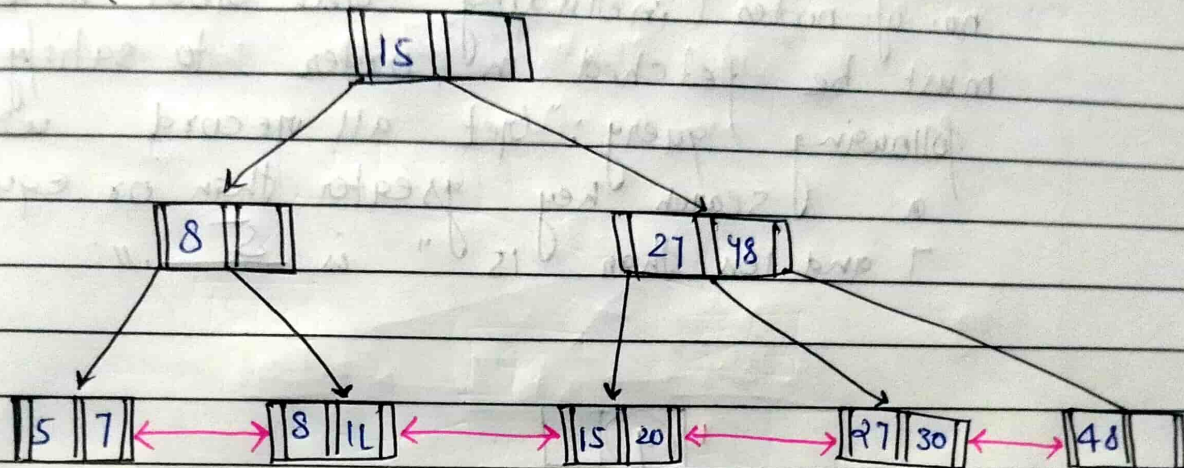
This structure makes the B+ Tree balanced, disk-efficient, and ideal for database indexing.

Internal node	leaf node
• Keys • Tree pointer	• Keys • Record pointer

Advantage :-

order = 3

leaf



leaf nodes are connected with linked list (singly/doubly).

Range queries take less time :-

Select * from Student where marks > 5 and marks < 27

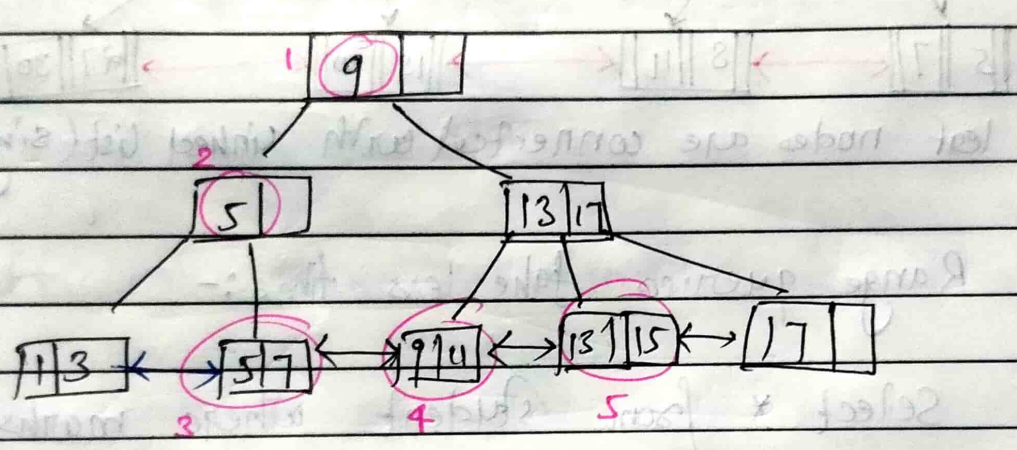
Properties :-

Same as in B-Tree :-

order = m

- Max^m children $\rightarrow m$
- Max^m key $\rightarrow m-1$
- Min^m key $\rightarrow \lceil m/2 \rceil - 1$

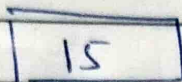
Q. With reference to the B+ Tree index of order 1 shown below, the min^m no. of nodes (including the root node) that must be fetched in order to satisfy the following query: "Get all record with a search key greater than or equal to 7 and less than 15" is 5.



→ Insertion:- { Same as we do in B-Tree }

Eg: 15, 8, ~~22~~, 5, 10, 18, 30, 95 of order 4.

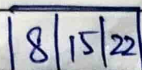
15



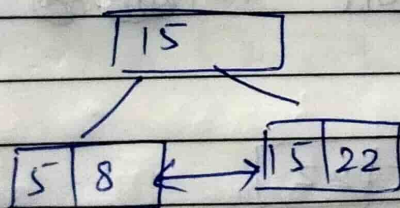
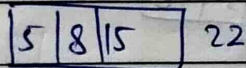
8



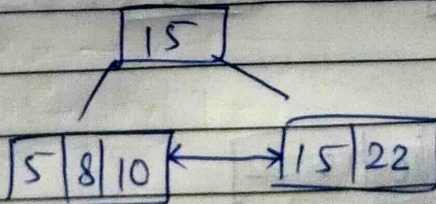
22



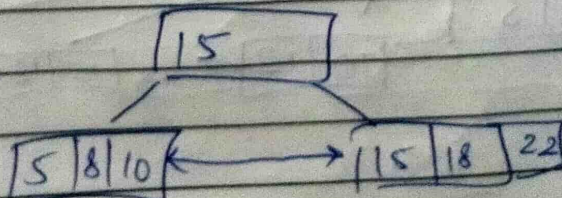
5



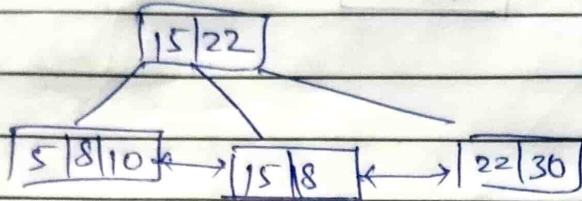
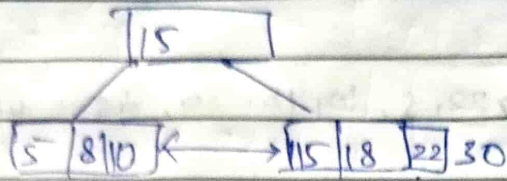
10



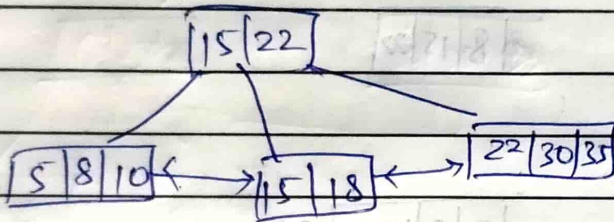
18



30 -

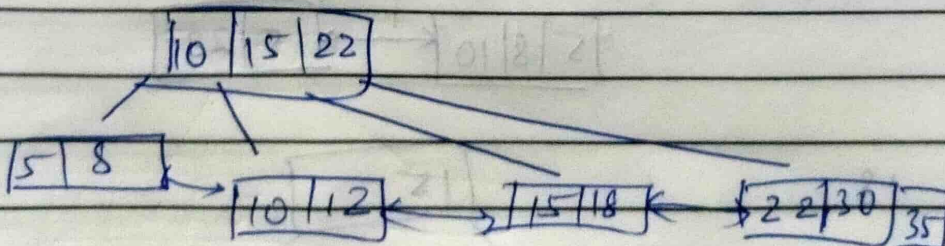
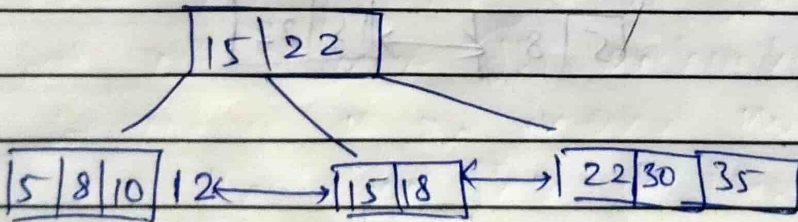


35 -



here Now insert 12

12 -

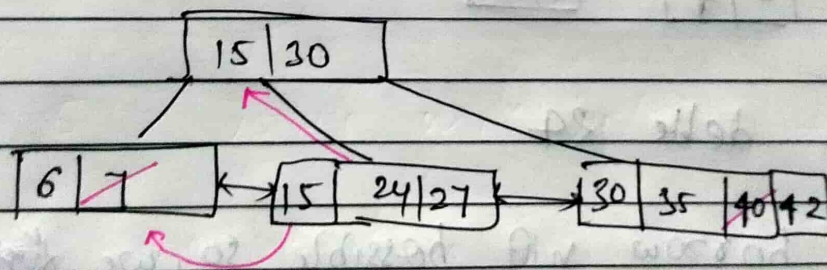


→ Deletion Operation:-

- After deletion no violation of min key, then no change in tree.
- If violation of min key, then borrow from sibling and update the anchor.
- If borrow from sibling can't possible then merge the node with sibling node. Either remove the anchor key (merge on leaf) or pull down the anchor key from parent (merge on internal node). Violation through parent.

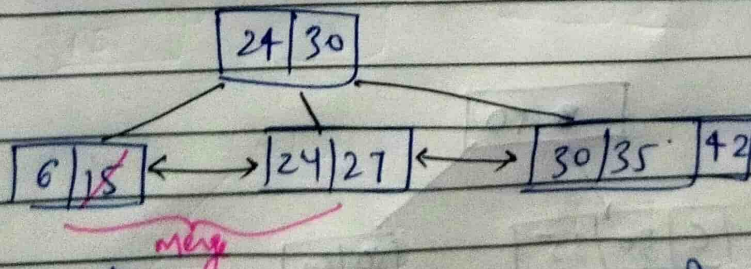
Eg: order- 5.

minⁿ key = 2.

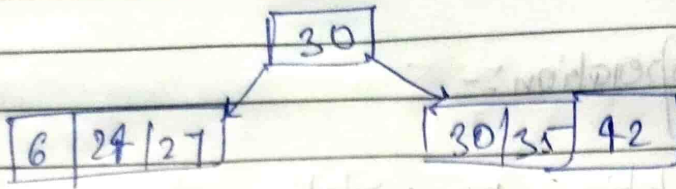


• delete 40, No violation.

Now, delete 7, violation.

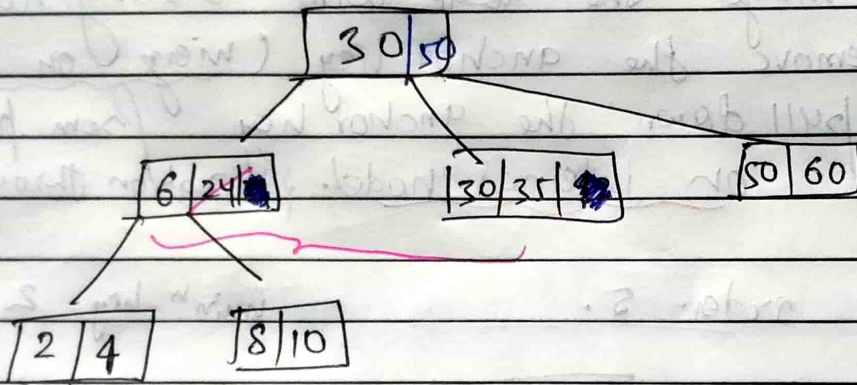


Now delete 15, borrow not possible, so we merge.



Remove the anchor because we remove the leaf node.

Eg: Order $\rightarrow 5$



delete 24

borrow not possible so, we ~~not~~ merge but not remove the anchor key because it is an internal node.
(notation through parent)

